



IF3270 Pembelajaran Mesin

Recurrent Neural Network (ANN)

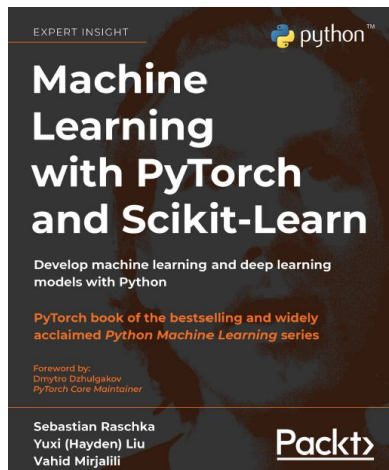
Tim Pengajar IF3270



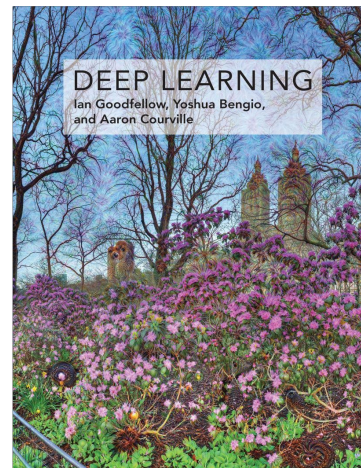
Review

- Machine learning (ML) overview
- Ensemble Methods → Supervised Learning
- Supervised Learning: Perceptron
- Supervised Learning: ANN: Feed Forward Neural Network
- Supervised Learning: ANN: Convolutional Neural Network
- Supervised Learning: ANN: Recurrent Neural Network → Today

References

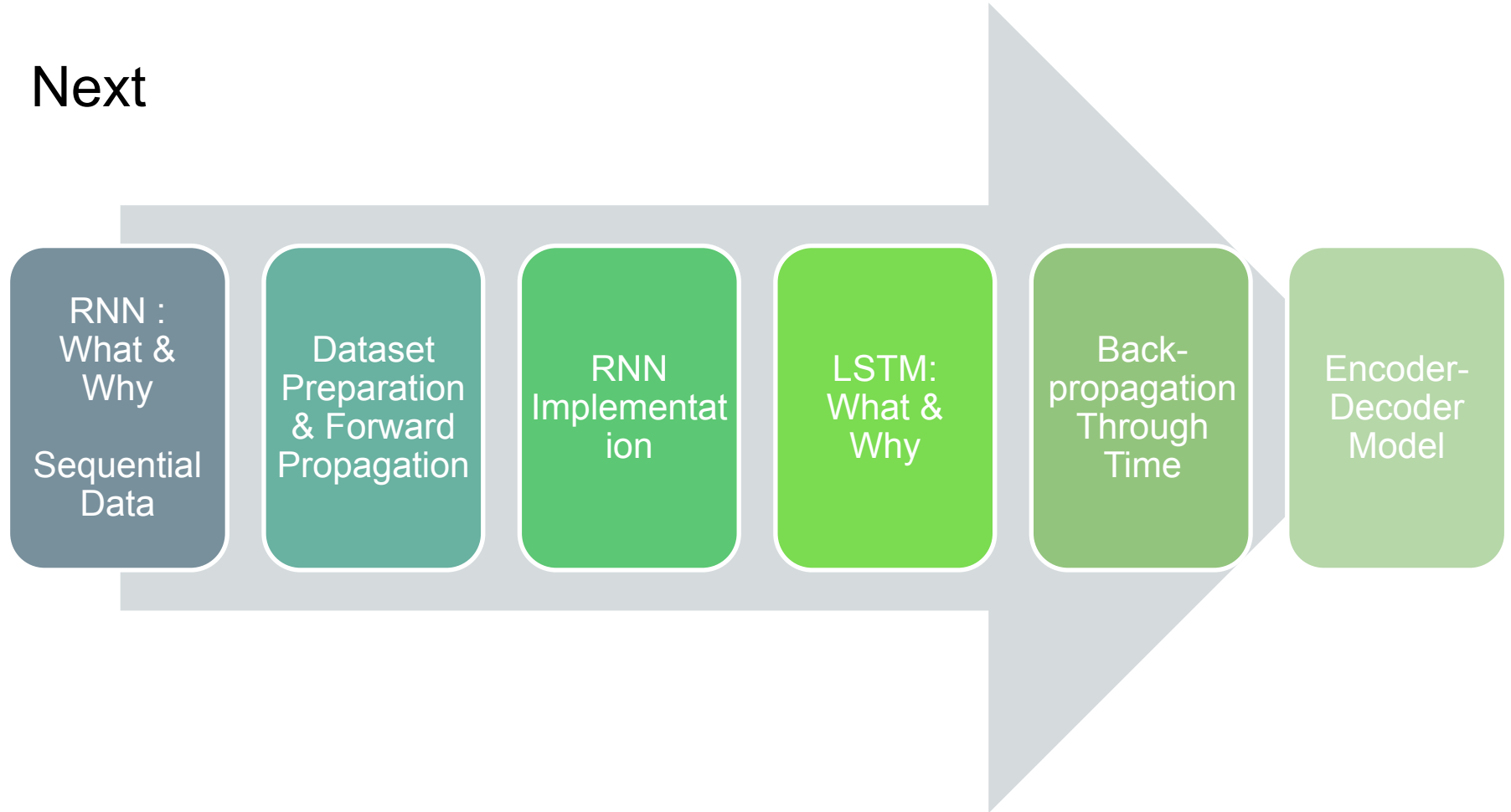


Raschka, et.al., Machine Learning with Pytorch and Scikit-Learn, Packt Publishing Ltd., 2022 (Chapter 15)

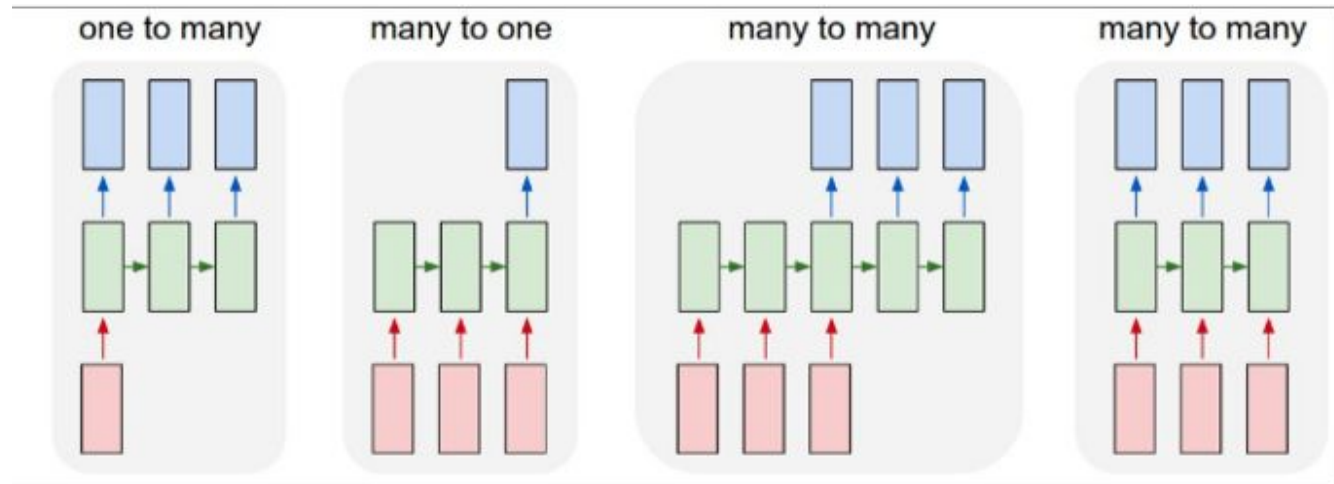
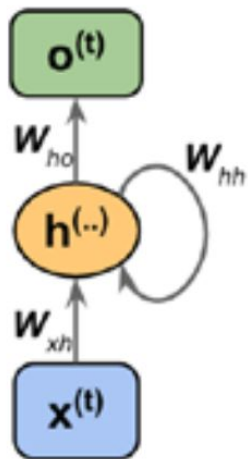


Deep learning. I Goodfellow, Y Bengio, A Courville, Y Bengio. MIT press 1 (2), 2016 (Chapter 10)

Next



Overview RNN



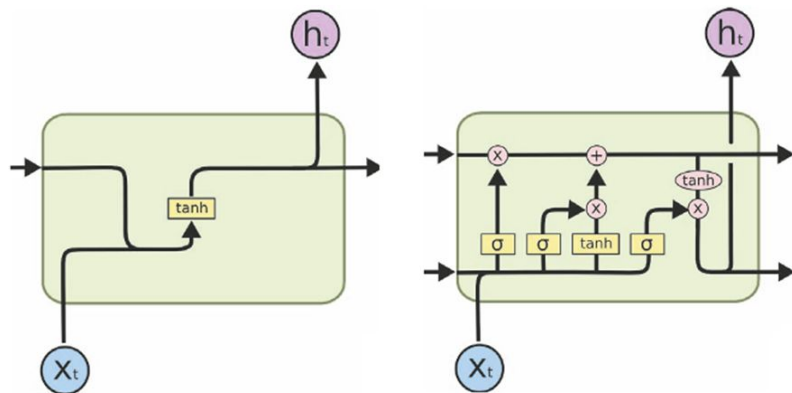
$$h^{(t)} = f_h(W_{xh}x^{(t)} + W_{hh}h^{(t-1)} + b_h)$$

$$o^{(t)} = f_y(W_{ho}h^{(t)} + b_{ho})$$

LSTM: What & Why

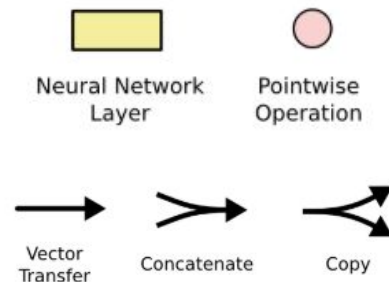
Long Short Term Memory (LSTM) Networks

- Introduced by [Hochreiter & Schmidhuber \(1997\)](#), LSTMs are explicitly designed to avoid the long-term dependency problem.
- Special kind of RNN. In standard RNNs, repeating module will have a very simple structure, such as a single tanh layer. Repeating module in an LSTM contains **four interacting layers**.

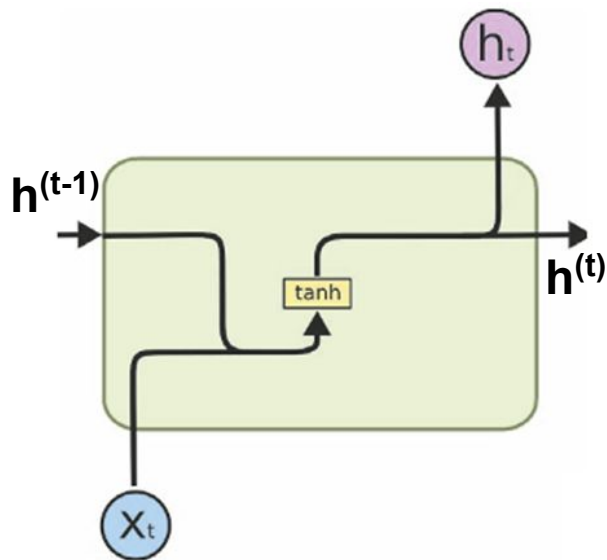


(a) RNN

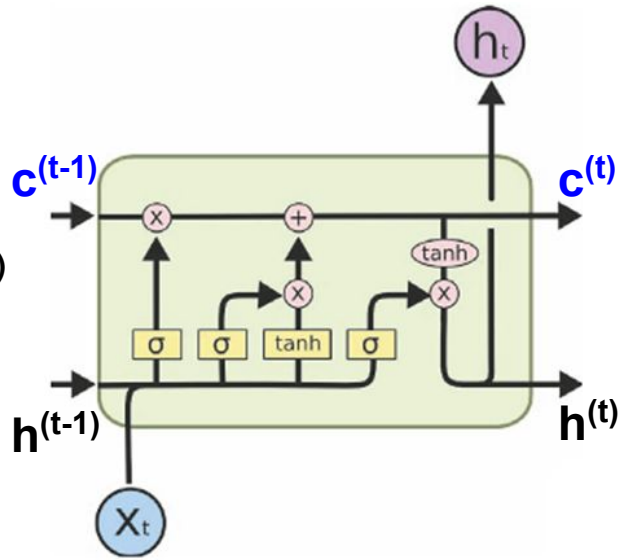
(b) LSTM



RNN vs LSTM



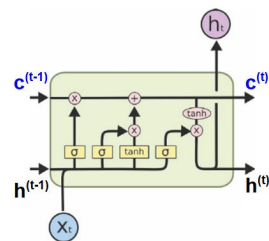
(a) RNN



(b) LSTM

$$h^{(t)} = f_h(W_{xh}x^{(t)} + W_{hh}h^{(t-1)} + b_h)$$

LSTM: Cell State & Gates



Cell State (C_t)

- as the “memory” of the network
- act as a transport highway that transfers relevant information throughout the processing of the sequence.

Forget Gate

- decides what information should be thrown away or kept.
- Values closer to 0 means to forget, and closer to 1 means to keep.

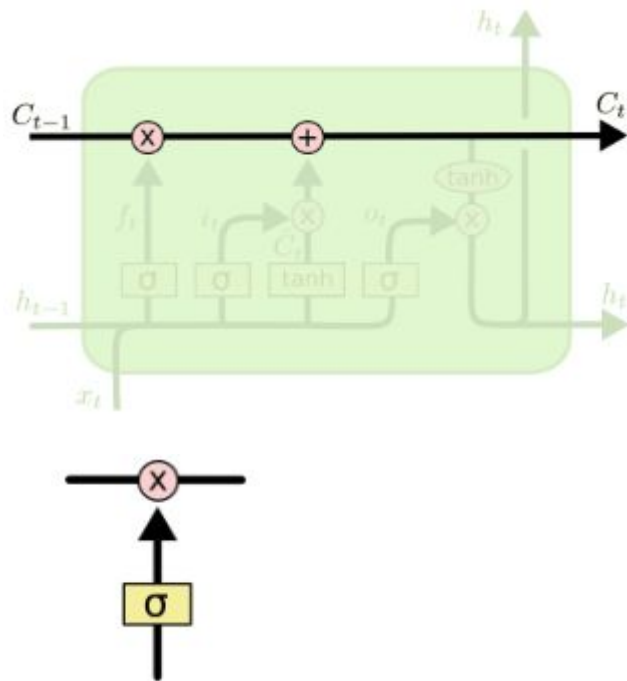
Input Gate

- Decides what information is relevant to add from the current step
- update the cell state by hidden state and current input

Output Gate

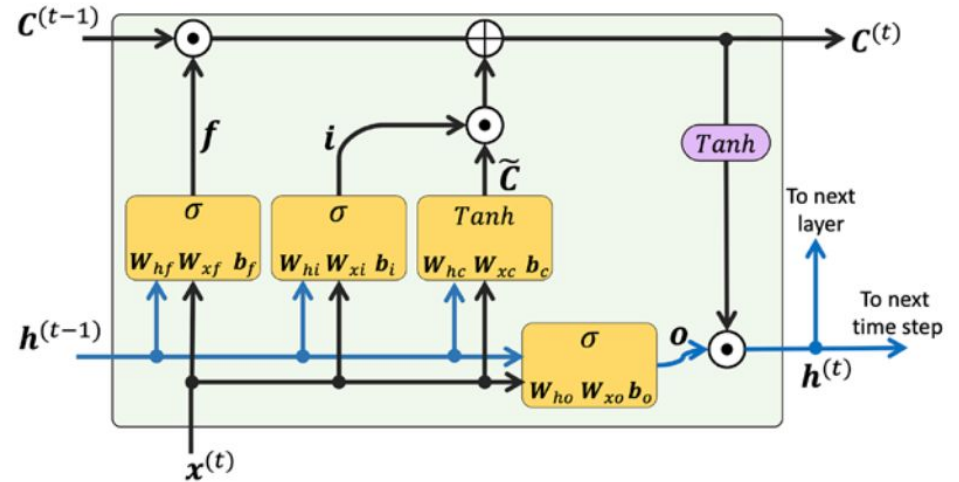
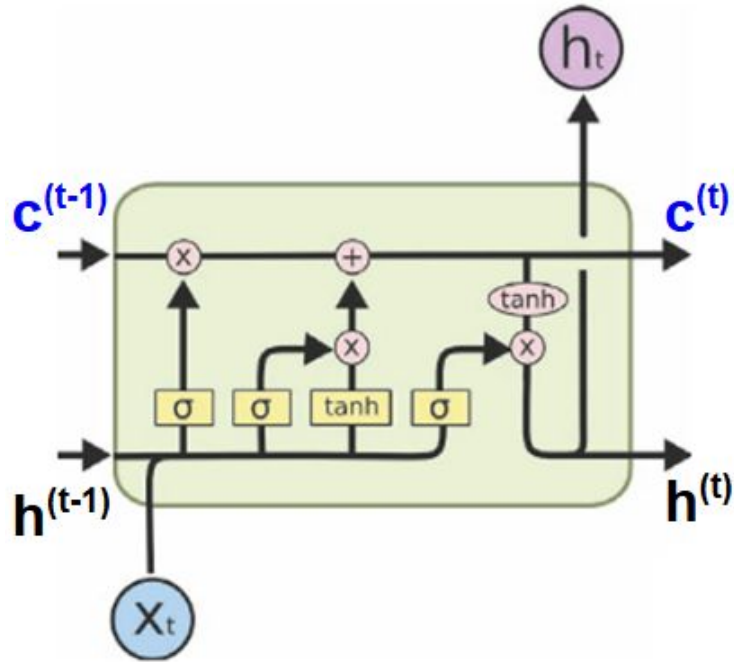
- decides what the next hidden state should be.
- Hidden state contains information on previous inputs. The hidden state is also used for predictions.

LSTM Unit



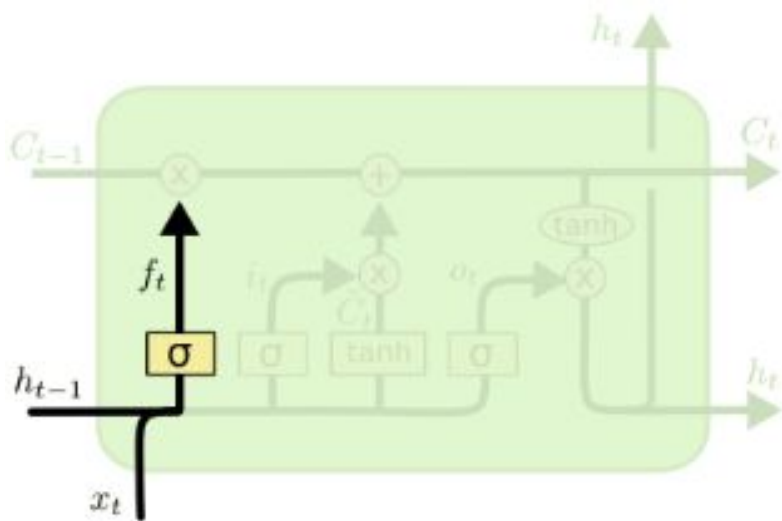
- Cell state $c^{(t)}$ is kind of like a conveyor belt. It runs straight down the entire chain, with only some minor linear interactions. It's very easy for information to just flow along it unchanged.
- Gates are a way to optionally let information through. The sigmoid layer outputs numbers between zero and one, describing how much of each component should be let through. A value of zero means “let nothing through,” while a value of one means “let everything through!”

LSTM: Notation



LSTM Unit: Forget Gate Layer

Forget gate layer: to decide what information we're going to **throw away** from the cell state. It looks at $h^{(t-1)}$ and $x^{(t)}$, and outputs a number between 0 and 1 for each number in the cell state $C^{(t-1)}$.



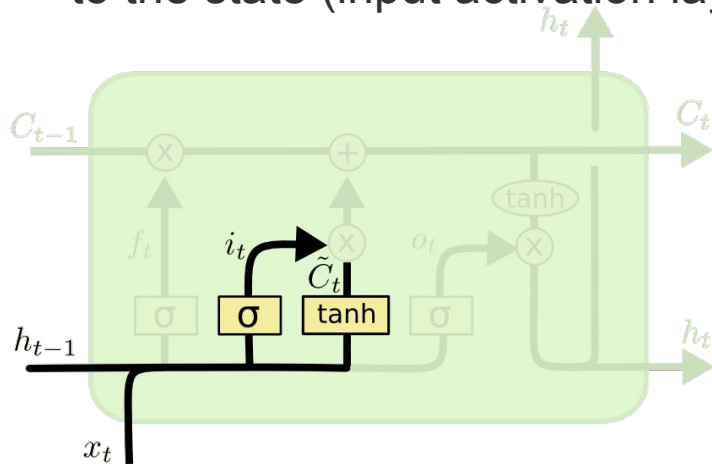
$$f_t = \sigma(W_{xf}x^{(t)} + W_{hf}h^{(t-1)} + b_f) = \sigma(W_f[h^{(t-1)}, x^{(t)}] + b_f)$$

1 represents “completely keep this” while a 0 represents “completely get rid of this.”

LSTM Unit: Input Gate & Tanh Layer

To decide what new information we're going to store in the cell state.

- Input gate layer decides which values we'll update.
- Tanh layer creates a vector of new candidate values, $\tilde{C}_t$, that could be added to the state (input activation layer)



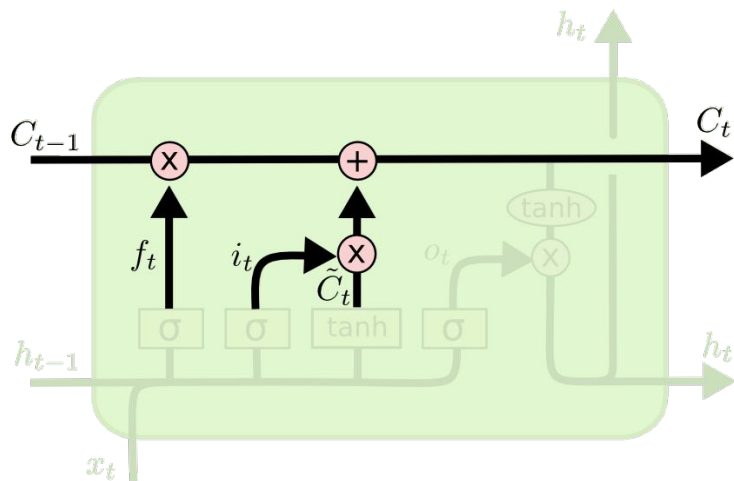
$$i_t = \sigma(W_{xi}x^{(t)} + W_{hi}h^{(t-1)} + b_i) = \sigma(W_i[h^{(t-1)}, x^{(t)}] + b_i)$$

$$\begin{aligned}\hat{C}_t &= \tanh(W_{xc}x^{(t)} + W_{hc}h^{(t-1)} + b_c) \\ &= \tanh(W_c[h^{(t-1)}, x^{(t)}] + b_c)\end{aligned}$$

LSTM Unit: Update $C^{(t-1)}$ into $C^{(t)}$

Multiply $C_{t-1} * f_t$, forgetting the things we decided to forget earlier.

Add $i_t * C_t$, the new candidate values, scaled by how much we decided to update each state value.



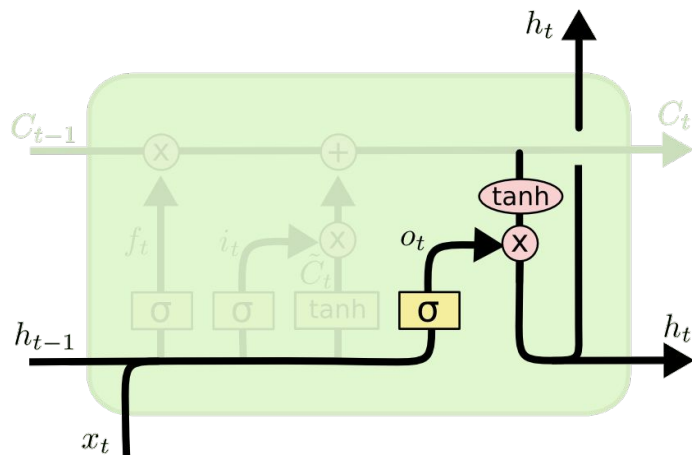
$$C^{(t)} = (C^{(t-1)} \odot f_t) \oplus (i_t \odot \hat{C}_t)$$

$\odot$: Hadamard product (element wise multiplication)

$\oplus$: element wise addition

LSTM Unit: Output h_t

Output h_t will be based on C_t , but will be a filtered version. Run a sigmoid layer which decides what parts of the cell state going to output. Then, put C_t through $\tanh$ (to push the values to be between -1 and 1) and multiply it by the output of the sigmoid gate, so that we only output the parts we decided to.

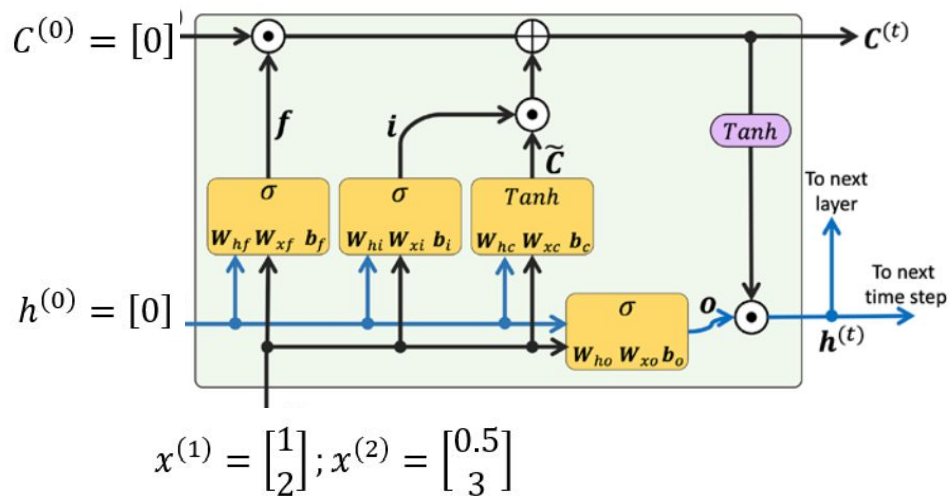


$$o_t = \sigma(W_{xo}x^{(t)} + W_{ho}h^{(t-1)} + b_o) = \sigma$$

$$(W_o[h^{(t-1)}, x^{(t)}] + b_o)$$

$$h^{(t)} = o_t \odot \tanh(C^{(t)})$$

Example



$$W_{xf} = \begin{bmatrix} 0.7 \\ 0.45 \end{bmatrix}; W_{hf} = [0.1]; b_f = [0.15]$$

$$W_{xi} = \begin{bmatrix} 0.95 \\ 0.8 \end{bmatrix}; W_{hi} = [0.8]; b_i = [0.65]$$

$$W_{xo} = \begin{bmatrix} 0.6 \\ 0.4 \end{bmatrix}; W_{ho} = [0.25]; b_o = [0.1]$$

$$W_{xc} = \begin{bmatrix} 0.45 \\ 0.25 \end{bmatrix}; W_{hc} = [0.15]; b_c = [0.2]$$

$$f_t = \sigma(W_{xf} x^{(t)} + W_{hf} h^{(t-1)} + b_f)$$

$$i_t = \sigma(W_{xi} x^{(t)} + W_{hi} h^{(t-1)} + b_i)$$

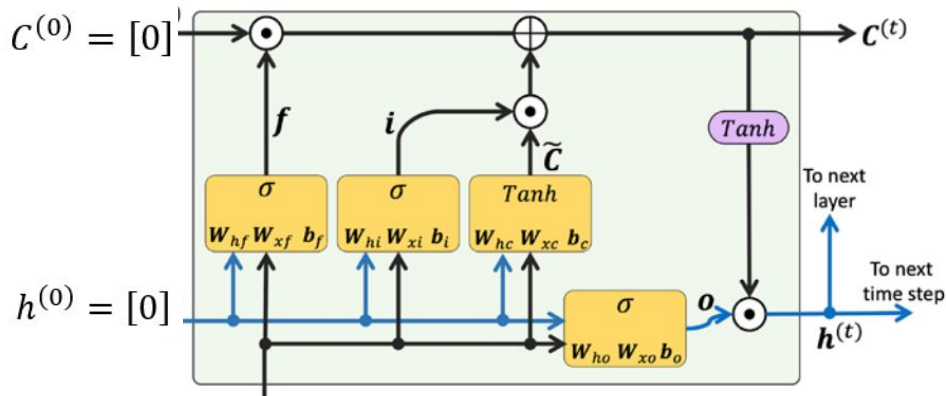
$$\hat{C}_t = \tanh(W_{xc} x^{(t)} + W_{hc} h^{(t-1)} + b_c)$$

$$C^{(t)} = (C^{(t-1)} \odot f_t) \odot \hat{C}_t$$

$$o_t = \sigma(W_{xo} x^{(t)} + W_{ho} h^{(t-1)} + b_o)$$

$$h^{(t)} = o_t \odot \tanh(C^{(t)})$$

Forward Propagation: Timestep t1



$$x^{(1)} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}; x^{(2)} = \begin{bmatrix} 0.5 \\ 3 \end{bmatrix}$$

$$f_t = \sigma(W_{xf} x^{(t)} + W_{hf} h^{(t-1)} + b_f)$$

$$i_t = \sigma(W_{xi} x^{(t)} + W_{hi} h^{(t-1)} + b_i)$$

$$\tilde{c}_t = \tanh(W_{xc} x^{(t)} + W_{hc} h^{(t-1)} + b_c)$$

$$c^{(t)} = (c^{(t-1)} \odot f_t) \oplus (i_t \odot \tilde{c}_t)$$

$$o_t = \sigma(W_{xo} x^{(t)} + W_{ho} h^{(t-1)} + b_o)$$

$$h^{(t)} = o_t \odot \tanh(c^{(t)})$$

$$f_1 = \sigma \left(\begin{bmatrix} 0.7 & 0.45 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} + \begin{bmatrix} 0.1 \end{bmatrix} [0] + \begin{bmatrix} 0.15 \end{bmatrix} \right) = \sigma([1.6] + [0] + [0.15]) = \sigma([1.75]) = [0.852]$$

$$i_1 = \sigma \left(\begin{bmatrix} 0.95 & 0.8 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} + \begin{bmatrix} 0.8 \end{bmatrix} [0] + \begin{bmatrix} 0.65 \end{bmatrix} \right) = \sigma([2.55] + [0] + [0.65]) = \sigma([3.2]) = [0.961]$$

$$\tilde{c}_1 = \tanh \left(\begin{bmatrix} 0.45 & 0.25 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} + \begin{bmatrix} 0.15 \end{bmatrix} [0] + \begin{bmatrix} 0.2 \end{bmatrix} \right) = \tanh([0.95] + [0] + [0.2]) = \tanh([1.15]) = [0.818]$$

$$c^{(1)} = ([0] \odot [0.852]) \oplus ([0.961] \odot [0.818]) = [0] \oplus [0.786] = [0.786]$$

$$o_1 = \sigma \left(\begin{bmatrix} 0.6 & 0.4 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} + \begin{bmatrix} 0.25 \end{bmatrix} [0] + \begin{bmatrix} 0.1 \end{bmatrix} \right) = \sigma([1.4] + [0] + [0.1]) = \sigma([1.5]) = [0.818]$$

$$h^{(1)} = [0.818] \odot \tanh[0.786] = [0.818] \odot [0.656] = [0.537]$$

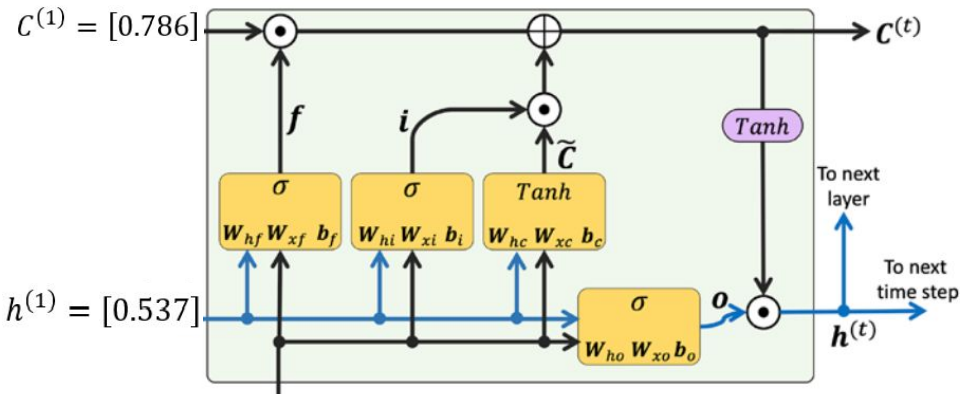
$$W_{xf} = \begin{bmatrix} 0.7 \\ 0.45 \end{bmatrix}; W_{hf} = [0.1]; b_f = [0.15]$$

$$W_{xi} = \begin{bmatrix} 0.95 \\ 0.8 \end{bmatrix}; W_{hi} = [0.8]; b_i = [0.65]$$

$$W_{xo} = \begin{bmatrix} 0.6 \\ 0.4 \end{bmatrix}; W_{ho} = [0.25]; b_o = [0.1]$$

$$W_{xc} = \begin{bmatrix} 0.45 \\ 0.25 \end{bmatrix}; W_{hc} = [0.15]; b_c = [0.2]$$

Forward Propagation: Timestep t2



$$W_{xf} = \begin{bmatrix} 0.7 \\ 0.45 \end{bmatrix}; W_{hf} = [0.1]; b_f = [0.15]$$

$$W_{xi} = \begin{bmatrix} 0.95 \\ 0.8 \end{bmatrix}; W_{hi} = [0.8]; b_i = [0.65]$$

$$W_{xo} = \begin{bmatrix} 0.6 \\ 0.4 \end{bmatrix}; W_{ho} = [0.25]; b_o = [0.1]$$

$$W_{xc} = \begin{bmatrix} 0.45 \\ 0.25 \end{bmatrix}; W_{hc} = [0.15]; b_c = [0.2]$$

$$x^{(1)} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}; x^{(2)} = \begin{bmatrix} 0.5 \\ 3 \end{bmatrix}$$

$$f_t = \sigma(W_{xf} x^{(t)} + W_{hf} h^{(t-1)} + b_f)$$

$$i_t = \sigma(W_{xi} x^{(t)} + W_{hi} h^{(t-1)} + b_i)$$

$$\hat{c}_t = \tanh(W_{xc} x^{(t)} + W_{hc} h^{(t-1)} + b_c)$$

)

$$C^{(t)} = (C^{(t-1)} \odot f_t) \oplus (i_t \odot \hat{c}_t)$$

$$o_t = \sigma(W_{xo} x^{(t)} + W_{ho} h^{(t-1)} + b_o)$$

$$h^{(t)} = o_t \odot \tanh(C^{(t)})$$

$$f_2 = \sigma \left(\begin{bmatrix} 0.7 & 0.45 \end{bmatrix} \begin{bmatrix} 0.5 \\ 3 \end{bmatrix} + [0.1][0.537] + [0.15] \right) = \sigma([1.7] + [0.0537] + [0.15]) = \sigma([1.904]) = [0.870]$$

$$i_2 = \sigma \left(\begin{bmatrix} 0.95 & 0.8 \end{bmatrix} \begin{bmatrix} 0.5 \\ 3 \end{bmatrix} + [0.8][0.537] + [0.65] \right) = \sigma([2.875] + [0.4296] + [0.65]) = \sigma([3.955]) = [0.981]$$

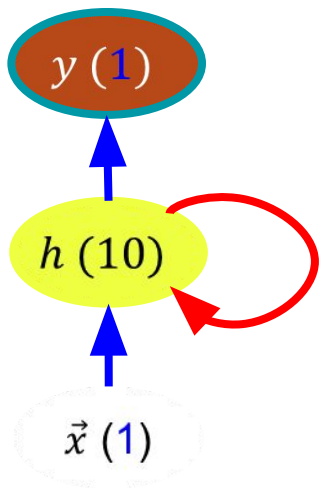
$$\tilde{c}_2 = \tanh \left(\begin{bmatrix} 0.45 & 0.25 \end{bmatrix} \begin{bmatrix} 0.5 \\ 3 \end{bmatrix} + [0.15][0.537] + [0.2] \right) = \tanh([0.975] + [0.081] + [0.2]) = \tanh([1.256]) = [0.850]$$

$$C^{(2)} = ([0.786] \odot [0.870]) \oplus ([0.981] \odot [0.850]) = [0.684] \oplus [0.834] = [1.518]$$

$$o_2 = \sigma \left(\begin{bmatrix} 0.6 & 0.4 \end{bmatrix} \begin{bmatrix} 0.5 \\ 3 \end{bmatrix} + [0.25][0.537] + [0.1] \right) = \sigma([1.5] + [0.134] + [0.1]) = \sigma([1.734]) = [0.850]$$

$$h^{(2)} = [0.850] \odot \tanh[1.518] = [0.850] \odot [0.908] = [0.772]$$

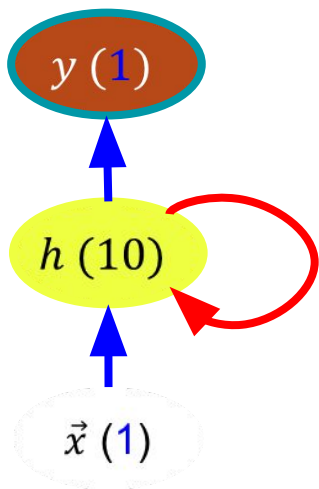
Implementing LSTM on Keras: Many to One



predict amazon stock closing prices, LSTM 50 timestep

```
from keras import Sequential
from keras.layers import LSTM, Dense
model = Sequential()
model.add(LSTM(10,
input_shape=(50,1))) #10 neurons &
process 50x1 sequences
model.add(Dense(1,activation='linear'))
#linear output as regression problem
```

Number of Parameter



```
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 10)	480
dense (Dense)	(None, 1)	11

Total params: 491

Trainable params: 491

Non-trainable params: 0

$$\text{Total parameter} = (1+10+1)*4*10+(10+1)*1=491$$

Simple RNN with equal networks: 131 parameter

W_{xh} : matrix hidden neurons x (input dimension + 1)

W_{hh} : matrix hidden neurons x hidden neurons

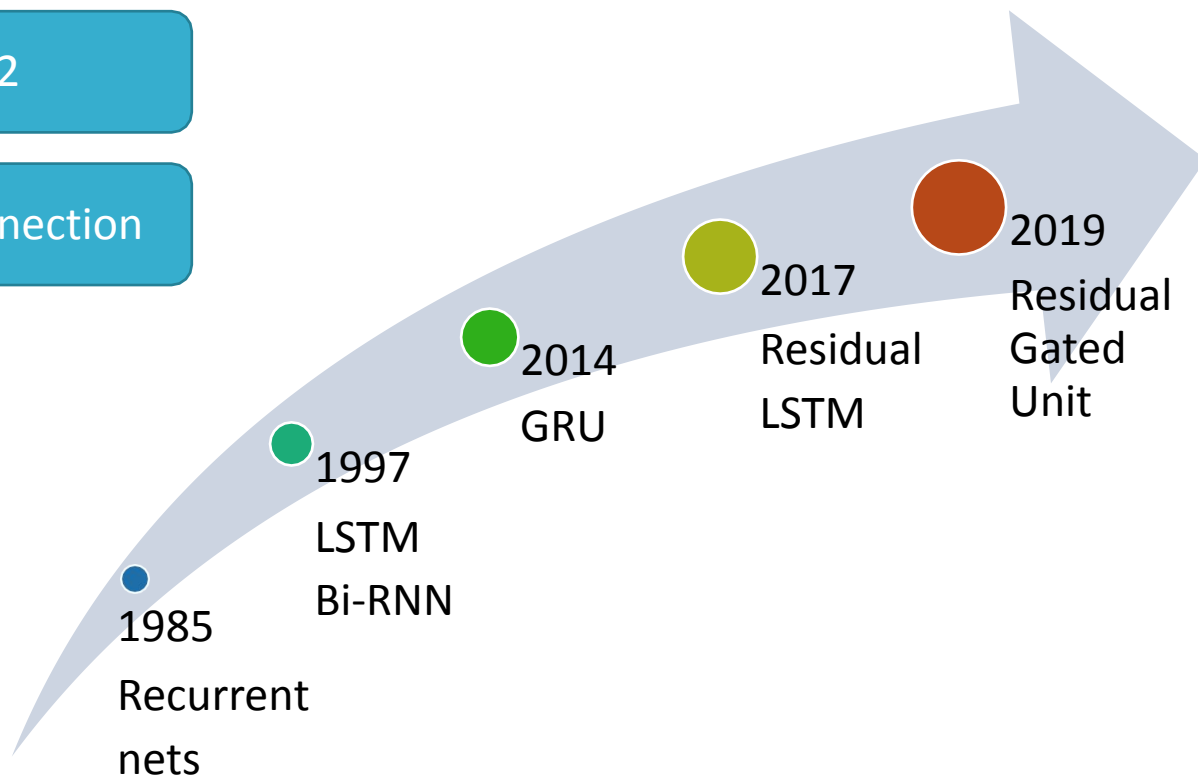
W_{hy} : matrix output neurons x (hidden neurons+1)

$$\begin{aligned} &\text{Total parameter for n unit lstm from m-} \\ &\text{dimension input to k dimension output} \\ &= (m+n+1)*4*n+(n+1)*k \end{aligned}$$

RNN $\rightarrow$ LSTM $\rightarrow$ GRU $\rightarrow$ ReGU

GRU: no cell state, 2
gates

ReGU: shortcut connection



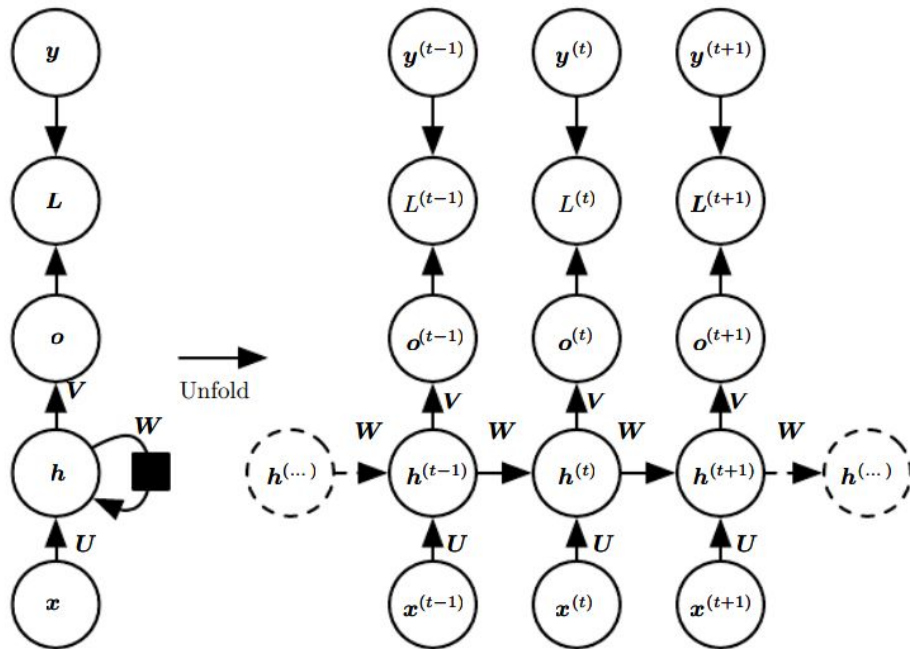
Backpropagation Through Time

Backpropagation Through Time (BPTT)

- BPTT learning algorithm is an extension of standard backpropagation that performs gradients descent on an **unfolded network**.
- The basic idea is that the overall loss, L , is the sum of all the loss functions at times $t = 1$ to $t = T$:

$$L = \sum_{t=1}^T L^{(t)}$$

$$\begin{aligned} \mathbf{U} &= \mathbf{W}_{xh} \\ \mathbf{W} &= \mathbf{W}_{hh} \\ \mathbf{V} &= \mathbf{W}_{hy} \end{aligned}$$



Cross Entropy Loss for One Instance

Example:

$$y=[1,0,0]; \hat{y}=[1,0,0]$$

$$L=-1.\ln(1)-0-0=0$$

$$L(y, \hat{y}) = - \sum_{k=1}^c y_k \ln(\hat{y}_k) = -y_{true} \ln(\hat{y}_{true})$$

y: one-hot encoded target

$$y=[1,0,0]; \hat{y}=[0.7,0.2,0.1]$$

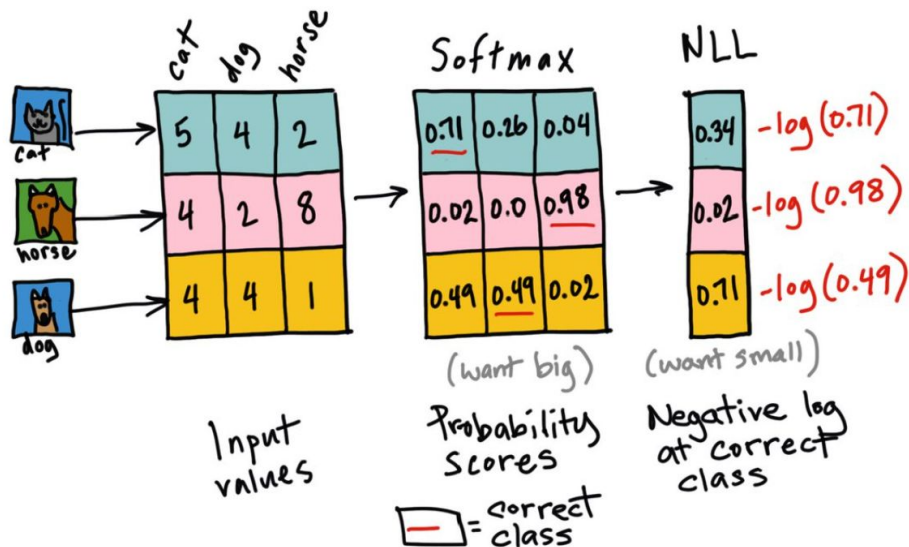
$$L=-1.\ln(0.7)-0.\ln(0.2)-0.\ln(0.1)=0.357$$

$$y=[0,1,0]; \hat{y}=[1,0,0]$$

$$L=-0.\ln(1)-1.\ln(0)-0=-1.(-\infty)=\infty$$

Cross Entropy Loss for Multiclass Classification

Negative Log Likelihood (NLL) Loss



$$L = \sum_{i=1}^n - \sum_{k=1}^c y_k \ln(\hat{y}_k) = \sum_{i=1}^n -y_{i,true} \ln(\hat{y}_{i,true})$$

y: one-hot encoded target

$y_{i,true}=1$, else $y_{i,j}=0$

$$L = -\ln(0.71) - \ln(0.98) - \ln(0.49)$$

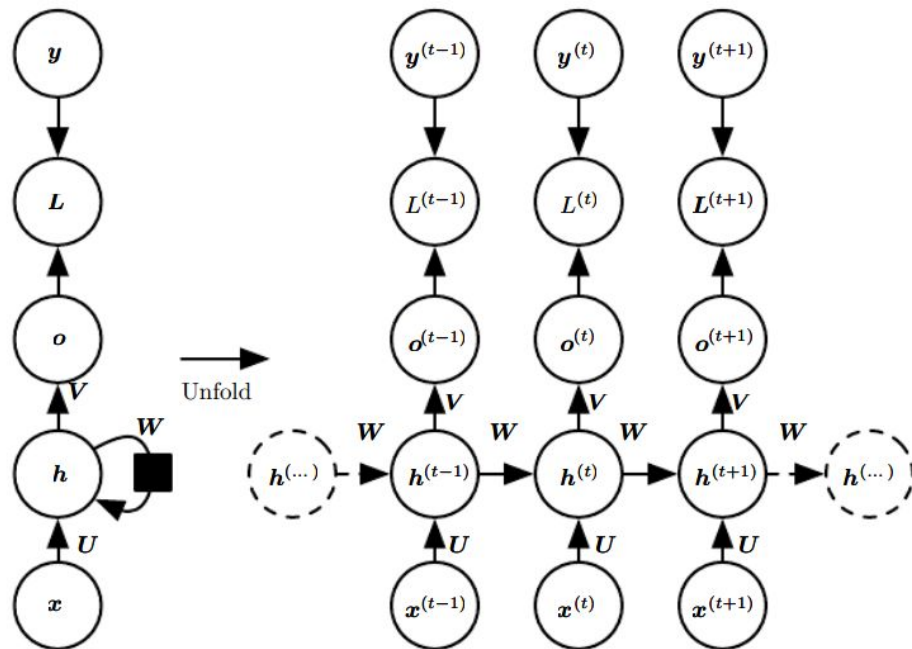
Cross Entropy Loss for Sequence Output

$$L^{(t)} = -y_{true}^{(t)} \ln(\hat{y}_{true}^{(t)})$$

$$L = \sum_{t=1}^T L^{(t)} = \sum_{t=1}^T -y_{true}^{(t)} \ln(\hat{y}_{true}^{(t)}) = \sum_{t=1}^T -\ln(\hat{y}_{true}^{(t)})$$

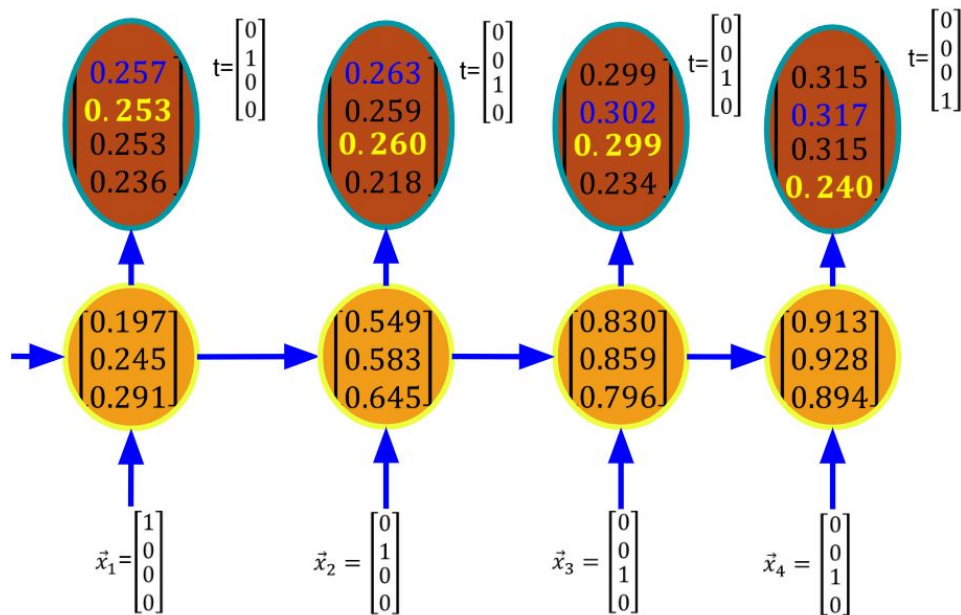
y : one-hot encoded target

$y_{true} = 1$, else $y_i = 0$



$$U = W_{xh} ; W = W_{hh} ; V = W_{hy}$$

Cross Entropy Loss for Sequence Output



$$L = \sum_{t=1}^T -y_{true}^{(t)} \ln(\hat{y}_{true}^{(t)})$$

$$L = -\ln(0.253) - \ln(0.26) - \ln(0.299) - \ln(0.240) = 5.356$$

Backpropagation Through Time (BPTT)

1. Forward pass: get sequence current output

$$\mathbf{h}^{(t)} = \mathbf{f}_h(\mathbf{W}_{xh}\mathbf{x}^{(t)} + \mathbf{W}_{hh}\mathbf{h}^{(t-1)} + \mathbf{b}_h) \quad \mathbf{o}^{(t)} = \mathbf{f}_y(\mathbf{W}_{ho}\mathbf{h}^{(t)} + \mathbf{b}_{ho})$$

2. Backward pass:

- a. Compute cross entropy error (negative log-likelihood of y) using the sequence current output ($\tilde{y} = \tilde{y}_1.. \tilde{y}_n$) and the sequence actual output ($y = y_1..y_n$):

$$L = \sum_{t=1}^T L^{(t)} = \sum_{t=1}^T -y_{true}^{(t)} \ln(\hat{y}_{true}^{(t)}) \quad L^{(t)} = -y_{true}^{(t)} \ln(\hat{y}_{true}^{(t)})$$

- b. For the unrolled network, the gradient is calculated for each time step with respect to the weight parameter
3. Now that the weight is the same for all the time steps the gradients can be combined together for all time steps. The weights are then updated for both recurrent neuron and the dense layers

BPTT for RNN

$$\frac{\partial L^{(t)}}{\partial W_{hy}}$$

$$L = \sum_{t=1}^T L^{(t)} = \sum_{t=1}^T -y_{true}^{(t)} \ln(\hat{y}_{true}^{(t)}) = \sum_{t=1}^T -\ln(\hat{y}_{true}^{(t)})$$

Negative log likelihood (CE loss)

$$\frac{\partial L^{(t)}}{\partial W_{hy}} = \frac{\partial L^{(t)}}{\partial \hat{y}^{(t)}} \frac{\partial \hat{y}^{(t)}}{\partial net^{(t)}} \frac{\partial net^{(t)}}{\partial W_{hy}} = \left(\hat{y}_{true}^{(t)} - y_{true}^{(t)} \right) h^{(t)}$$

$$\frac{\partial L^{(t)}}{\partial \hat{y}^{(t)}} \frac{\partial \hat{y}^{(t)}}{\partial net^{(t)}} = \hat{y}_{true}^{(t)} - y_{true}^{(t)} = \begin{cases} \hat{y}_j^{(t)} - 1 = -(1 - \hat{y}_j^{(t)}); j = targetClass \\ \hat{y}_j^{(t)} - 0 = \hat{y}_j^{(t)}; j \neq targetClass \end{cases}$$

CNN slides:

$$\frac{\partial E_d}{\partial net_j} = \begin{cases} p_j, j \neq targetClass \\ -(1 - p_j), j = targetClass \end{cases}$$

$$\frac{\partial net^{(t)}}{\partial W_{hy}} = h^{(t)}$$

RNN slides: $net^{(t)} = \mathbf{W}_{hy} \mathbf{h}^{(t)} + \mathbf{b}_{hy}$ and $\hat{\mathbf{y}}^{(t)} = \mathbf{f}_y(\mathbf{W}_{ho} \mathbf{h}^{(t)} + \mathbf{b}_{ho})$

BPTT for RNN

$$\frac{\partial L^{(t)}}{\partial W_{hh}}$$

$$\frac{\partial L^{(t)}}{\partial W_{hh}} = \frac{\partial L^{(t)}}{\partial \hat{y}^{(t)}} \frac{\partial \hat{y}^{(t)}}{\partial net^{(t)}} \frac{\partial net^{(t)}}{\partial h^{(t)}} \sum_{k=1}^t \frac{\partial h^{(t)}}{\partial h^{(k)}} \frac{\partial h^{(k)}}{\partial neth^{(k)}} \frac{\partial neth^{(k)}}{\partial W_{hh}} = (\hat{y}_{true}^{(t)} - y_{true}^{(t)}) W_{hy} \sum_{k=1}^t \frac{\partial h^{(t)}}{\partial h^{(k)}} \cdot f'_h \cdot h^{(k-1)}$$

$$\frac{\partial L^{(t)}}{\partial \hat{y}^{(t)}} \frac{\partial \hat{y}^{(t)}}{\partial net^{(t)}} = \hat{y}_{true}^{(t)} - y_{true}^{(t)} = \begin{cases} \hat{y}_j^{(t)} - 1 = -(1 - \hat{y}_j^{(t)}); j = targetClass \\ \hat{y}_j^{(t)} - 0 = \hat{y}_j^{(t)}; j \neq targetClass \end{cases}$$

$$\frac{\partial net^{(t)}}{\partial h^{(t)}} = W_{hy} \quad \text{RNN slides: } net^{(t)} = \mathbf{W}_{hy} \mathbf{h}^{(t)} + \mathbf{b}_{hy} \text{ and } \hat{y}^{(t)} = f_y(\mathbf{W}_{ho} \mathbf{h}^{(t)} + \mathbf{b}_{ho})$$

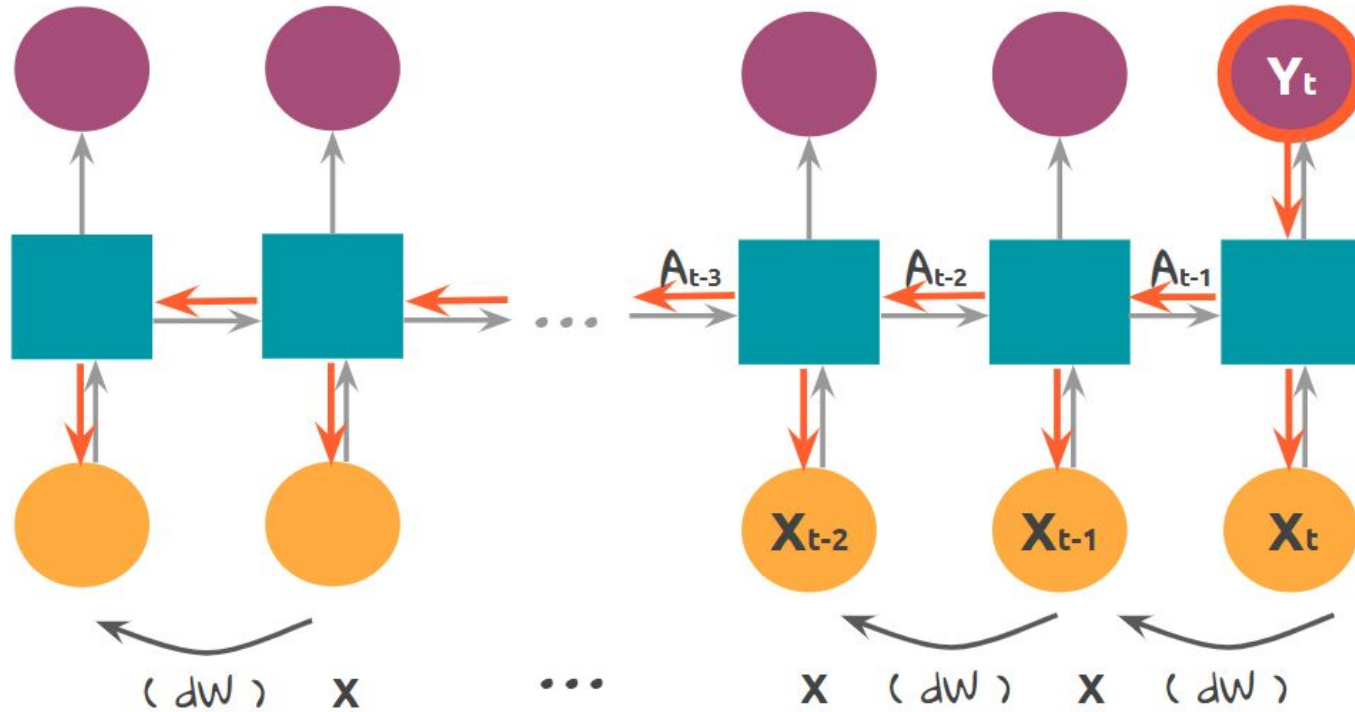
$$\frac{\partial h^{(k)}}{\partial neth^{(k)}} = f'_h \quad \text{RNN slides: } \mathbf{h}^{(t)} = f_h(\mathbf{W}_{xh} \mathbf{x}^{(t)} + \mathbf{W}_{hh} \mathbf{h}^{(t-1)} + \mathbf{b}_h)$$

Here, $\frac{\partial \mathbf{h}^{(t)}}{\partial \mathbf{h}^{(k)}}$ is computed as a multiplication of adjacent time steps:

$$\frac{\partial \mathbf{h}^{(t)}}{\partial \mathbf{h}^{(k)}} = \prod_{i=k+1}^t \frac{\partial \mathbf{h}^{(i)}}{\partial \mathbf{h}^{(i-1)}}$$

Raschka, et.al., Machine Learning with Pytorch and Scikit-Learn, Packt Publishing Ltd., 2022

Backward Pass



BPTT for LSTM

BPTT: Update Weight

$$V(t+1) = V(t) + \alpha \frac{\partial E}{\partial V}$$

$$\frac{\partial E}{\partial V} = s(t) \cdot \delta_o(t)$$

$$\frac{\partial E}{\partial a_o(t)} = \frac{\partial E}{\partial o(t)} \frac{\partial o(t)}{\partial a_o(t)} = \delta_o(t) = (d(t) - o(t)) f'_o(a_o(t))$$

$$U(t+1) = U(t) + \alpha \frac{\partial E}{\partial U}$$

$$\frac{\partial E}{\partial U} = x(t) \cdot \delta_h(t)$$

$$\frac{\partial E}{\partial a_h(t)} = \delta_h(t) = f'_h(a_h(t)) \odot (V^T \delta_o(t) + W^T \delta_h(t+1))$$

$$W(t+1) = W(t) + \alpha \frac{\partial E}{\partial W}$$

$$\frac{\partial E}{\partial W} = s(t-1) \cdot \delta_h(t)$$

f'_o, f'_h : turunan fungsi aktivasi di output/hidden layer

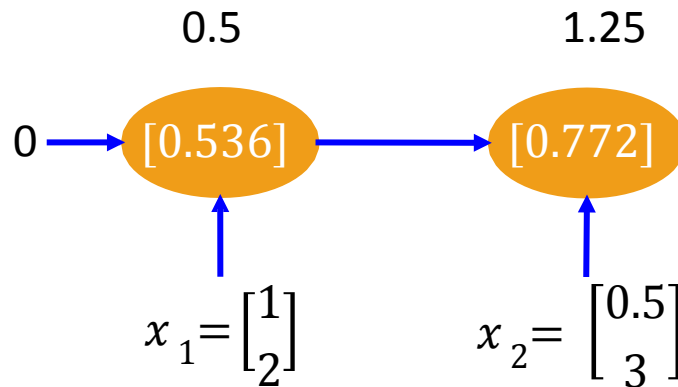
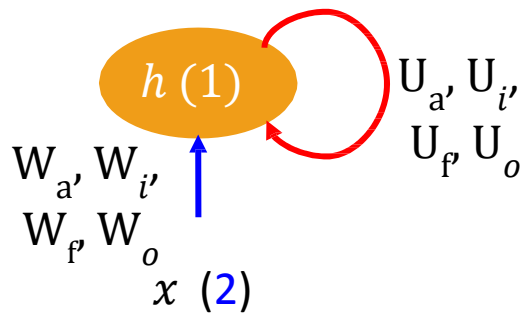
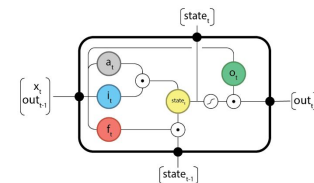
$\delta_o(t)$: error di output layer

$\delta_h(t)$: error di hidden layer

$\odot$: Hadamard product

$s(t)$: hasil $f_h(a_h(t))$

Example: Forward Pass

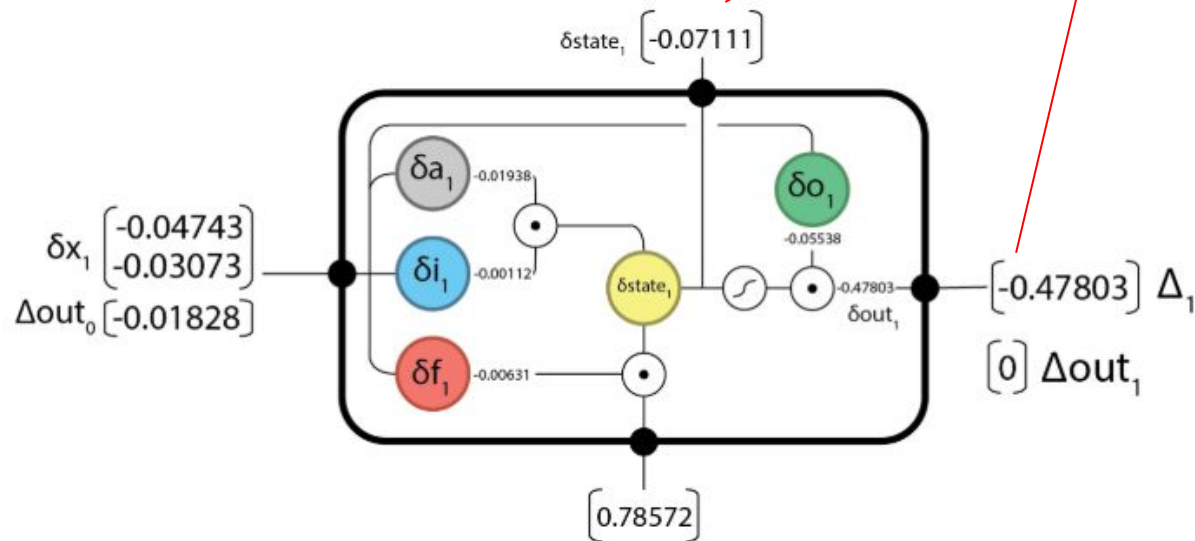


W transpose			
0.45	0.95	0.7	0.6
0.25	0.8	0.45	0.4
U transpose			
0.15	0.8	0.1	0.25

t=1	t=0
a1	a0
0.8498	0.81755
i1	i0
0.98118	0.96083
f1	f0
0.8703	0.85195
o1	o0
0.84993	0.81757

t=1	t=0
state1	state0
1.5176	0.78752
out1	out0
0.77197	0.53631

Backward Phase t=1



$$\delta out_t = \Delta_t + \Delta out_t$$

$$\delta state_t = \delta out_t \odot o_t \odot (1 - \tanh^2(state_t)) + \delta state_{t+1} \odot f_{t+1}$$

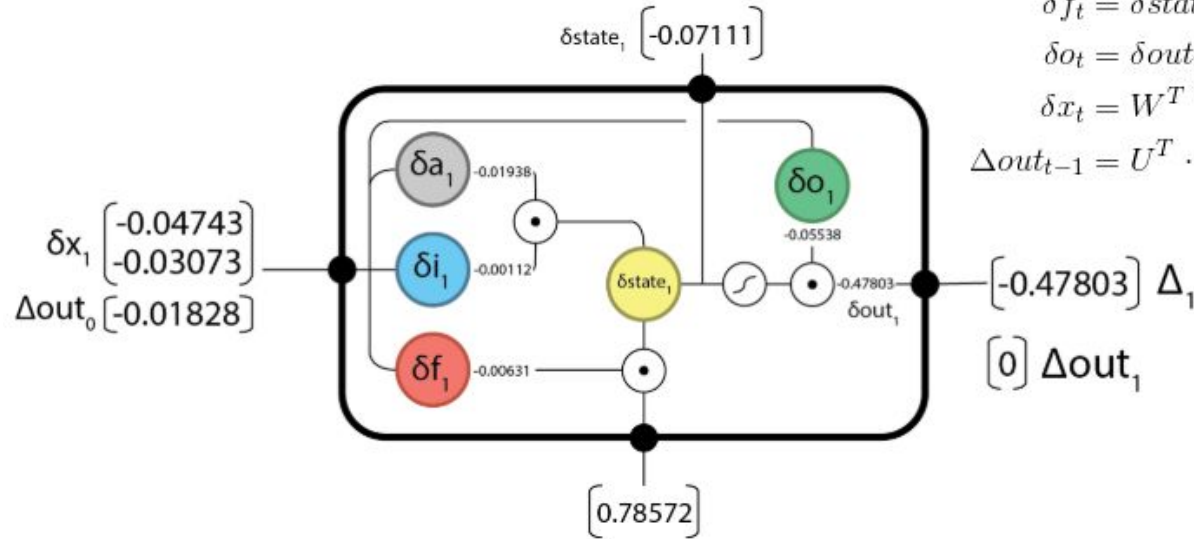
$$\Delta_1 = \partial_x E = 0.77197 - 1.25 = -0.47803$$

$\Delta out_1 = 0$ because there are no future time-steps.

$$\delta out_1 = \Delta_1 + \Delta out_1 = -0.47803 + 0 = -0.47803$$

$$\delta state_1 = \delta out_1 \odot o_1 \odot (1 - \tanh^2(state_1)) + \delta state_2 \odot f_2 = -0.47803 \times 0.84993 \times (1 - \tanh^2(1.5176)) + 0 \times 0 = -0.07111$$

Backward Phase t=1 (lanj)



$$\delta a_t = \delta state_t \odot i_t \odot (1 - a_t^2)$$

$$\delta i_t = \delta state_t \odot a_t \odot i_t \odot (1 - i_t)$$

$$\delta f_t = \delta state_t \odot state_{t-1} \odot f_t \odot (1 - f_t)$$

$$\delta o_t = \delta out_t \odot \tanh(state_t) \odot o_t \odot (1 - o_t)$$

$$\delta x_t = W^T \cdot \delta gates_t$$

$$\Delta out_{t-1} = U^T \cdot \delta gates_t$$

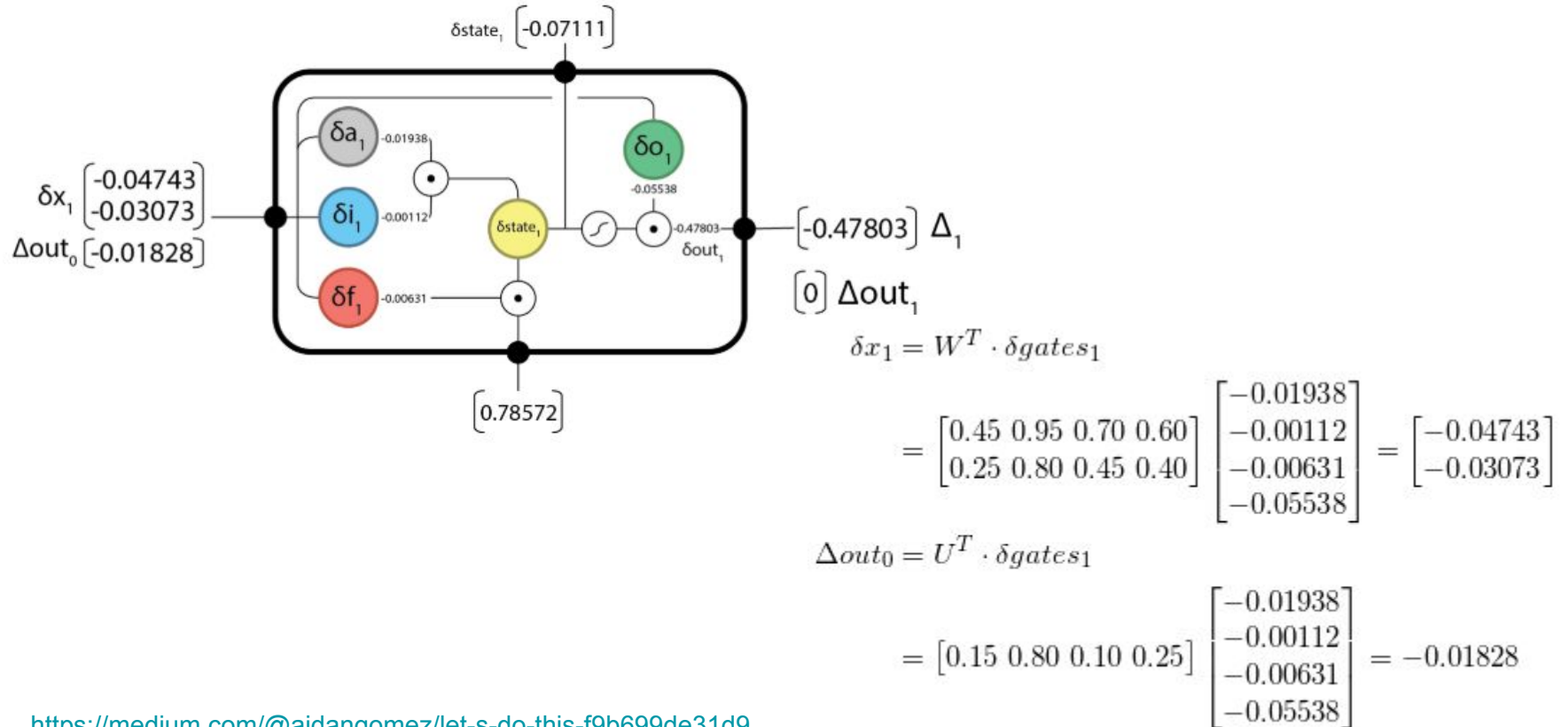
$$\delta a_1 = \delta state_1 \odot i_1 \odot (1 - a_1^2) = -0.07111 \times 0.98118 \times (1 - 0.84980^2) = -0.01938$$

$$\delta i_1 = \delta state_1 \odot a_1 \odot i_1 \odot (1 - i_1) = -0.07111 \times 0.84980 \times 0.98118 \times (1 - 0.98118) = -0.00112$$

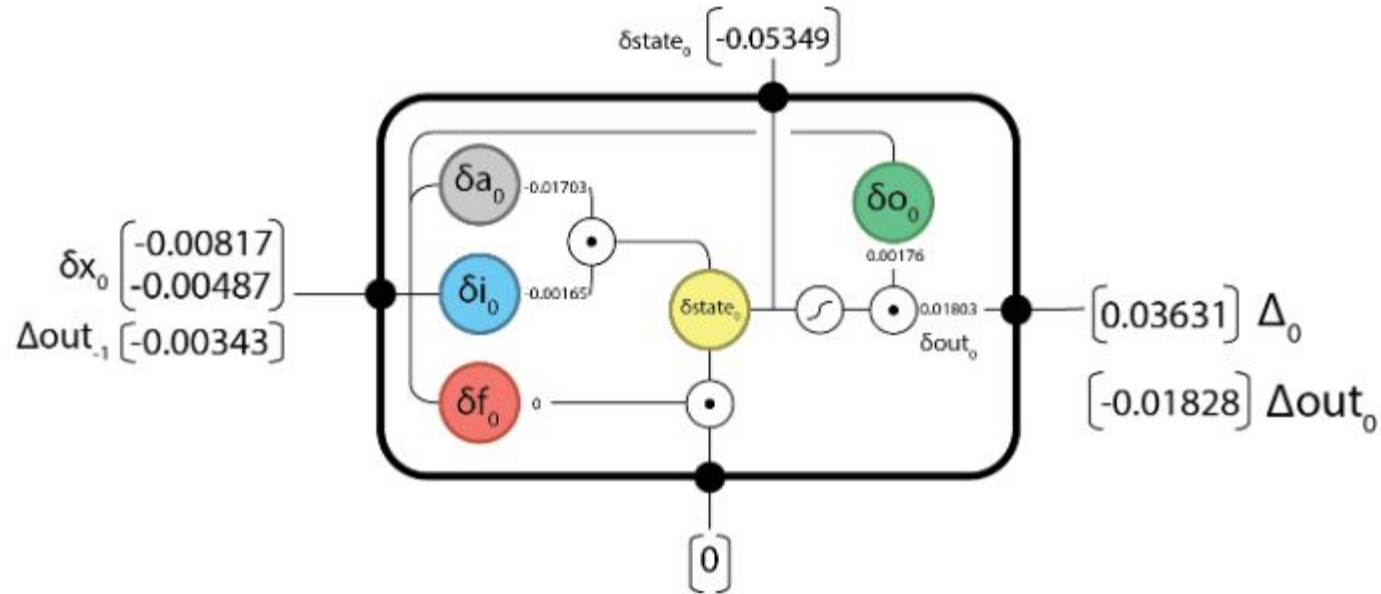
$$\delta f_1 = \delta state_1 \odot state_0 \odot f_1 \odot (1 - f_1) = -0.07111 \times 0.78572 \times 0.87030 \times (1 - 0.87030) = -0.00631$$

$$\delta o_1 = \delta out_1 \odot \tanh(state_1) \odot o_1 \odot (1 - o_1) = -0.47803 \times \tanh(1.5176) \times 0.84993 \times (1 - 0.84993) = -0.05538$$

Backward Phase t=1 (lanj)



Backward Phase t=0



Backward Phase t=0: Detil

$$\Delta_0 = \partial_x E = 0.53631 - 0.5 = 0.03631$$

$$\Delta_{out_0} = -0.01828, \text{ passed back from } T=1$$

$$\delta_{out_0} = \Delta_0 + \Delta_{out_0} = 0.03631 + -0.01828 = 0.01803$$

$$\delta_{state_0} = \delta_{out_0} \odot o_0 \odot (1 - \tanh^2(state_0)) + \delta_{state_1} \odot f_1 = 0.01803 \times 0.81757 \times (1 - \tanh^2(0.78572)) + -0.07111 \times 0.87030 = -0.05349$$

$$\delta a_0 = \delta_{state_0} \odot i_0 \odot (1 - a_0^2) = -0.05349 \times 0.96083 \times (1 - 0.81775^2) = -0.01703$$

$$\delta i_0 = \delta_{state_0} \odot a_0 \odot i_0 \odot (1 - i_0) = -0.05349 \times 0.81775 \times 0.96083 \times (1 - 0.96083) = -0.00165$$

$$\delta f_0 = \delta_{state_0} \odot state_{-1} \odot f_0 \odot (1 - f_0) = -0.05349 \times 0 \times 0.85195 \times (1 - 0.85195) = 0$$

$$\delta o_0 = \delta_{out_0} \odot \tanh(state_0) \odot o_0 \odot (1 - o_0) = 0.01803 \times \tanh(0.78572) \times 0.81757 \times (1 - 0.81757) = 0.00176$$

$$\delta x_0 = W^T \cdot \delta gates_0$$

$$= \begin{bmatrix} 0.45 & 0.95 & 0.70 & 0.60 \\ 0.25 & 0.80 & 0.45 & 0.40 \end{bmatrix} \begin{bmatrix} -0.01703 \\ -0.00165 \\ 0 \\ 0.00176 \end{bmatrix} = \begin{bmatrix} -0.00817 \\ -0.00487 \end{bmatrix}$$

$$\Delta_{out_{-1}} = U^T \cdot \delta gates_1$$

$$= \begin{bmatrix} 0.15 & 0.80 & 0.10 & 0.25 \end{bmatrix} \begin{bmatrix} -0.01703 \\ -0.00165 \\ 0 \\ 0.00176 \end{bmatrix} = -0.00343$$

Update Phase

$$\begin{aligned}\delta W &= \sum_{t=0}^T \delta gates_t \otimes x_t \\ &= \begin{bmatrix} -0.01703 \\ -0.00165 \\ 0 \\ 0.00176 \end{bmatrix} [1.0 \ 2.0] + \begin{bmatrix} -0.01938 \\ -0.00112 \\ -0.00631 \\ -0.05538 \end{bmatrix} [0.5 \ 3.0] = \begin{bmatrix} -0.02672 & -0.0922 \\ -0.00221 & -0.00666 \\ -0.00316 & -0.01893 \\ -0.02593 & -0.16262 \end{bmatrix}\end{aligned}$$

$$\begin{aligned}\delta U &= \sum_{t=0}^{T-1} \delta gates_{t+1} \otimes out_t \\ &= \begin{bmatrix} -0.01938 \\ -0.00112 \\ -0.00631 \\ -0.05538 \end{bmatrix} [0.53631] = \begin{bmatrix} -0.01039 \\ -0.00060 \\ -0.00338 \\ -0.02970 \end{bmatrix}\end{aligned}$$

$$\begin{aligned}\delta b &= \sum_{t=0}^T \delta gates_t \\ &= \begin{bmatrix} -0.01703 \\ -0.00165 \\ 0 \\ 0.00176 \end{bmatrix} + \begin{bmatrix} -0.01938 \\ -0.00112 \\ -0.00631 \\ -0.05538 \end{bmatrix} = \begin{bmatrix} -0.03641 \\ -0.00277 \\ -0.00631 \\ -0.05362 \end{bmatrix}\end{aligned}$$

$\lambda=0.1$

Wold

$$W_a = \begin{bmatrix} 0.45 \\ 0.25 \end{bmatrix}, U_a = [0.15], b_a = [0.2]$$

$$W_i = \begin{bmatrix} 0.95 \\ 0.8 \end{bmatrix}, U_i = [0.8], b_i = [0.65]$$

$$W_f = \begin{bmatrix} 0.7 \\ 0.45 \end{bmatrix}, U_f = [0.1], b_f = [0.15]$$

$$W_o = \begin{bmatrix} 0.6 \\ 0.4 \end{bmatrix}, U_o = [0.25], b_o = [0.1]$$

$$W^{new} = W^{old} - \lambda * \delta W^{old}$$

Wnew

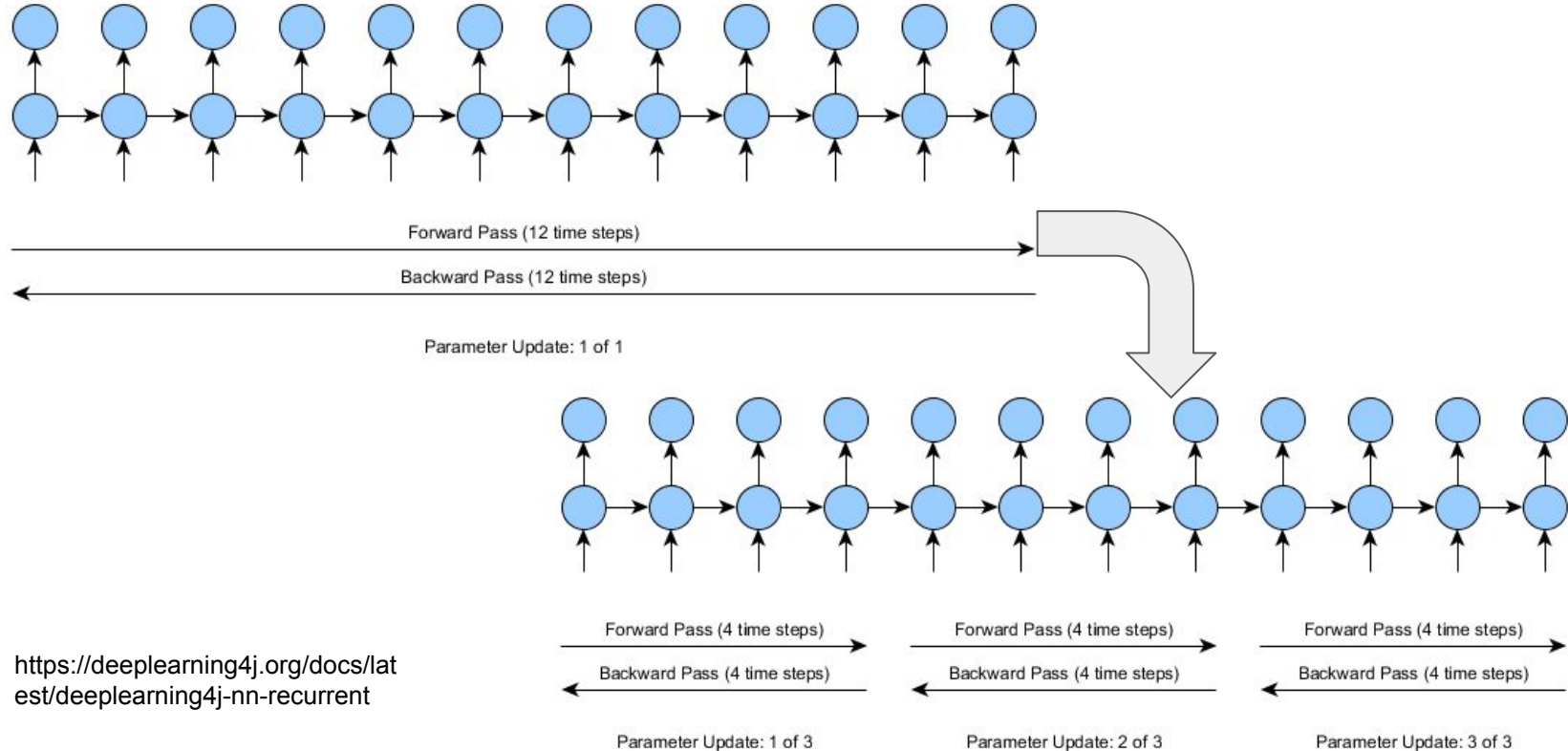
$$W_a = \begin{bmatrix} 0.45267 \\ 0.25922 \end{bmatrix}, U_a = [0.15104], b_a = [0.20364]$$

$$W_i = \begin{bmatrix} 0.95022 \\ 0.80067 \end{bmatrix}, U_i = [0.80006], b_i = [0.65028]$$

$$W_f = \begin{bmatrix} 0.70031 \\ 0.45189 \end{bmatrix}, U_f = [0.10034], b_f = [0.15063]$$

$$W_o = \begin{bmatrix} 0.60259 \\ 0.41626 \end{bmatrix}, U_o = [0.25297], b_o = [0.10536]$$

Truncated BPTT: Ilustrasi



Truncated BPTT

- Truncated BPTT was developed in order to **reduce the computational complexity** of each parameter update in a recurrent neural network.
- Truncated BPTT splits the forward and backward passes into a set of smaller forward/backward pass operations. The specific length of these forward/backward pass segments is a parameter set by the user.
- With a TBPTT length of 4. Suppose that at time step 10, the network needs to store some information from time step 0 in order to make an accurate prediction. In standard BPTT, this is ok: the gradients can flow backwards all the way along the unrolled network, from time 10 to time 0. In truncated BPTT, **this is problematic**: the gradients from time step 10 simply don't flow back far enough to cause the required parameter updates that would store the required information. This tradeoff is usually worth it, and (as long as the truncated BPTT lengths are set appropriately), truncated BPTT works well in practice.

Bidirectional RNNs: BPTT

```
for  $t = 1$  to  $T$  do
  Do forward pass for the forward hidden layer, storing activations at
  each timestep
for  $t = T$  to  $1$  do
  Do forward pass for the backward hidden layer, storing activations at
  each timestep
for  $t = 1$  to  $T$  do
  Do forward pass for the output layer, using the stored activations from
  both hidden layers
```

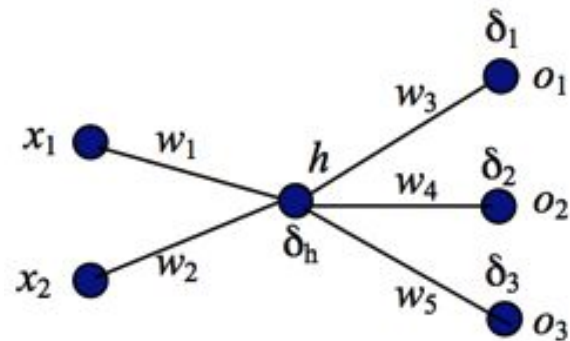
Algorithm 3.1: BRNN Forward Pass

```
for  $t = T$  to  $1$  do
  Do BPTT backward pass for the output layer only, storing  $\delta$  terms at
  each timestep
for  $t = T$  to  $1$  do
  Do BPTT backward pass for the forward hidden layer, using the stored
   $\delta$  terms from the output layer
for  $t = 1$  to  $T$  do
  Do BPTT backward pass for the backward hidden layer, using the
  stored  $\delta$  terms from the output layer
```

Algorithm 3.2: BRNN Backward Pass

Vanishing Gradient Problem

- Backpropagation computes gradients by the chain rule.
- In some cases, the gradient will be vanishingly small, effectively preventing the weight from changing its value.
 - In the worst case, this may completely stop the neural network from further training.
- This has the effect of multiplying n of these small numbers to compute gradients of the "front" layers in an n-layer network
 - the gradient (error signal) decreases exponentially with n.



$$\delta_1 = o_1 * (1 - o_1) * (t_1 - o_1);$$

$$w_3 = w_3 + lr * \delta_1 * h$$

$$\delta_h = h * (1 - h) * [(w_3 \delta_1 + w_4 \delta_2 + w_5 \delta_3)]$$

$$w_1 = w_1 + lr * \delta_h * x_1$$

Encoder – Decoder Model

Sequence-to-Sequence Model

- A sequence-to-sequence model aims to map a fixed-length input with a fixed-length output where the length of the input and output may differ.

- Use cases:

1. Machine translation

[English] Mary eats apples. -> [French]
Marie mange des pommes.

2. Question answering

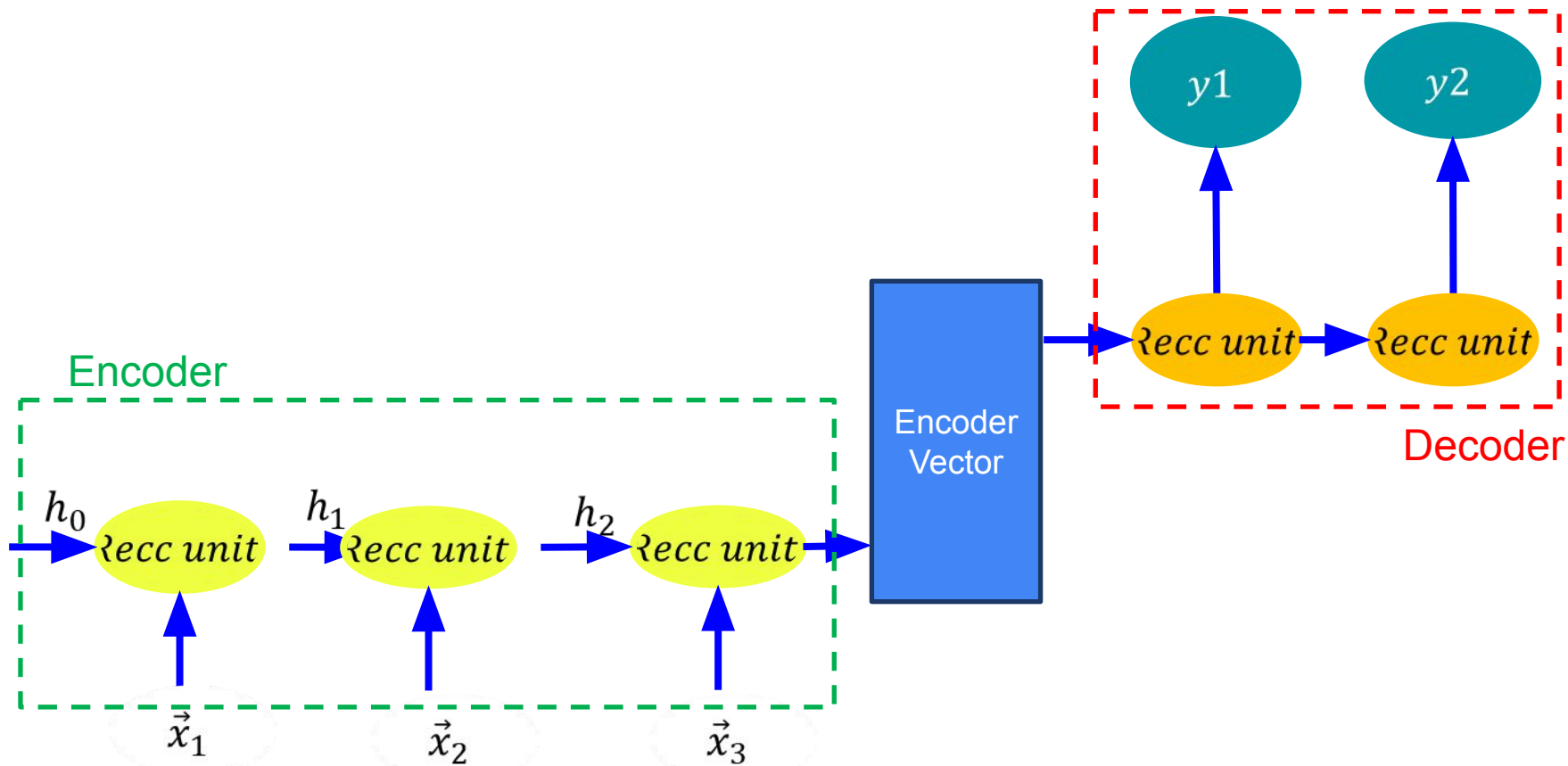
[Question] Tim is playing in his room.||Where is Tim? -> [Answer] Tim is in his room.

3. Video captioning:



S2VT: A herd of zebras are walking in a field.

RNN Encoder-Decoder



Encoder-Decoder

Encoder

- A stack of several recurrent units where each accepts a single element of the input sequence, collects information for that element and propagates it forward.

Encoder Vector

- This is the final hidden state produced from the encoder part of the model. It is calculated using the formula for Encoder.
- This vector aims to encapsulate the information for all input elements in order to help the decoder make accurate predictions.
- It acts as the initial hidden state of the decoder part of the model.

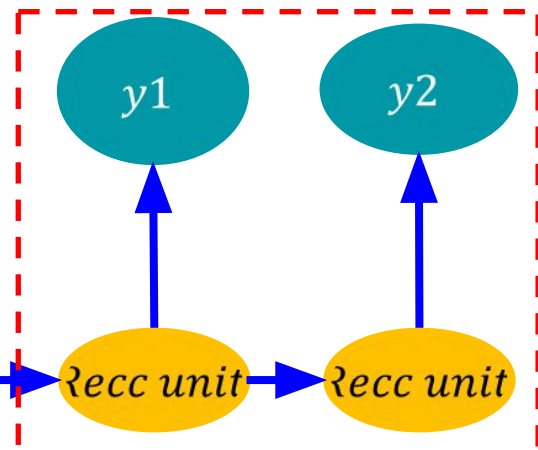
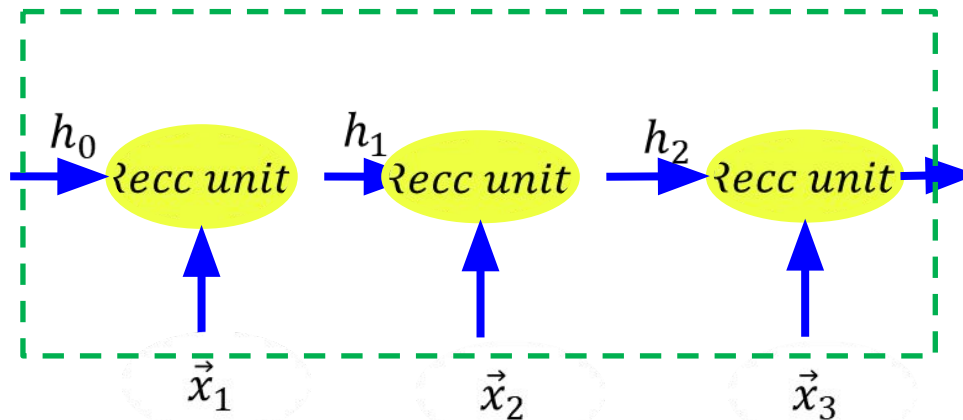
Decoder

- A stack of several recurrent units where each predicts an output y_t at a time step t .
- Each recurrent unit accepts a hidden state from the previous unit and produces an output as well as its own hidden state.

RNN Encoder-Decoder

$$h_t = f(W_{xh}x_t + W_{hh}h_{t-1})$$

Encoder

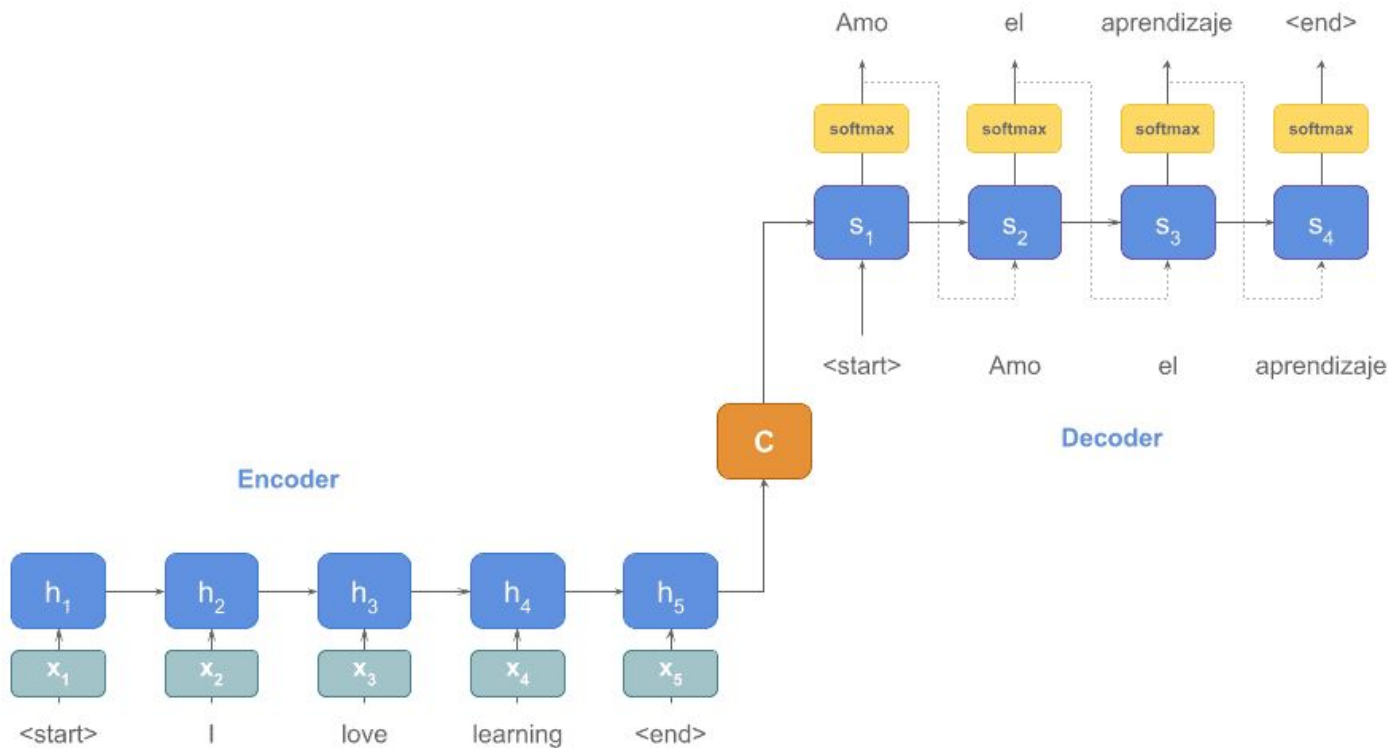


Decoder

$$h_t = f(W_{hh}h_{t-1})$$

$$y_t = \text{softmax}(W_{hv}h_t)$$

Example: Machine Translation



Thank you